

### Find ith Bit.

हमें find करना है 5th position पर कौन सा bit है।

8 7 6 5 4 3 2 1 0  
n → 1 0 0 1 1 0 1 0 1  
mask → 0 0 0 1 0 0 0 0 0

1 << i

n & mask → Non zero → 1  
0

### Set ith Bit.

1 Set करना है,

8 7 6 5 4 3 2 1 0  
n → 1 0 0 1 1 0 1 0 1  
mask → 0 0 0 0 0 1 0 0 0

mask = 1 << i

n = n | mask

### Clear ith Bit

0 करना है

8 7 6 5 4 3 2 1 0  
n → 1 0 0 1 1 0 1 0 1  
mask → 1 1 1 1 0 1 1 1 1

mask = ~(1 << i)

n ⇒ n & mask.

Find number of bits to change to convert a to b.

a → 1 0 1 1 0  
b → 1 1 0 1 1

Ans 3

1 नीचे 1 को Count कर मो वही Answer होगा।

Find no. of 1.

$a = 13$

Right shift करो ~~जाओ~~ और 1 के साथ & करने को और low करो जाओ तब तक कि 0 न हो जाये

0 0 0 0 0 1 0 0 0

no. of bits in any binary representation number

$n \rightarrow \log_2 n + 1$    
 decimal format

$n \rightarrow \log_{10} n + 1$    
 to find out no. of digits in decimal like 13.

2nd method.

बैट का चिह्न

$n \& (n-1)$

$13 \rightarrow 1101$

$$\begin{array}{r} 1101 \\ \& 1100 \\ \hline 1100 \\ \& 1011 \\ \hline 1000 \\ \& 0111 \\ \hline 0000 \end{array}$$

$$\begin{array}{c} 1101 \\ \downarrow \\ 1100 \\ \downarrow \\ 1000 \\ \downarrow \\ 0000 \end{array}$$

Ques) Find the only non-Repeating element in an array where every element repeats twice.

$a = [5, 4, 1, 4, 3, 5, 1]$

int res = 0;

res = 5 ^ 4 ^ 1 ^ 4

$5 \wedge 1 \wedge 3 \wedge 5 \Rightarrow 1 \wedge 3 \Rightarrow 3$  Ans

Ques) find the two non-repeating elements in an array where every element repeats twice.

$a = [5, 4, 1, 4, 3, 5, 1, 2]$

$res = 3 \wedge 2$

$$\begin{array}{r} 11 \\ 01 \\ \hline 0 \end{array}$$


$$\begin{array}{l} \swarrow 1 \\ 5, 1, 3, 5, 1 \\ \downarrow \\ 3 \wedge 2 \wedge 5 \wedge 1 \wedge 3 \wedge 5 \wedge 1 \end{array}$$

$$\searrow 0 \\ 4, 4, 2$$

$3 \wedge 2 \wedge 5 \wedge 1 \wedge 3 \wedge 5 \wedge 1$

2 ✓

$2 \wedge 3 \wedge 2$

3 ✓

3. find the only non-repeating element in an array where every other element repeats thrice.

LeetCode 137

$a = [2, 2, 1, 5, 1, 1, 2]$

$int\ ones = 0; \ twos = 0; \ threes = 0;$

$for (int\ i = 0; i < nums.length; i++) \{$

$\quad twos = ones \oplus nums[i];$

$\quad ones = nums[i];$

$\quad threes = ones \oplus twos;$

$\quad ones = \sim threes;$

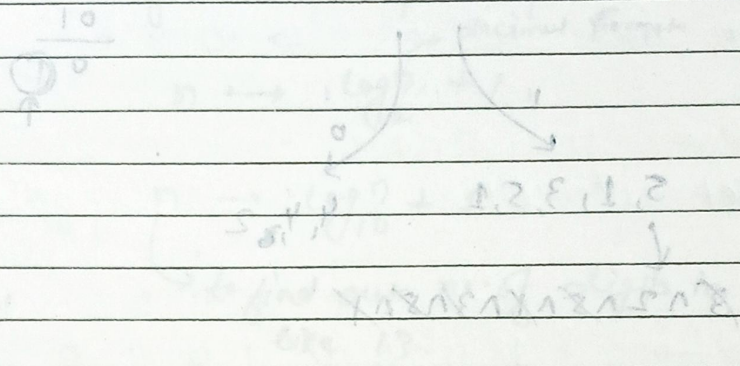
$\quad twos = \sim threes;$

$\}$

$\quad return\ ones;$

To check whether given number 'n' is power of 2 or not.

```
public boolean isPowerOfTwo(int n) {
    return (n & (n-1)) == 0 && n > 0;
```



3. find the only element in an array which appears every other element twice.

```
a = [2, 2, 1, 2, 1, 2, 1, 2, 1, 2]
```

if over = 0, floor = 0, there = 0

for (int i = 0; i < nums.length; i++)

if (over & floor(i) == 1)

over = 1;

floor(i) = 1;

over = 0;

floor(i) = 0;

}

return over;

# Anur Bhaiya DSA

- \* In an sorted array after rotation one half part is sorted and other part is unsorted.

Sieve of Eratosthenes.

|           |   |   |   |   |   |   |   |   |   |    |    |    |
|-----------|---|---|---|---|---|---|---|---|---|----|----|----|
|           | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| boolean[] | F | T | T | T | T | T | T | T | T | T  | T  | T  |
|           | F |   |   | F |   | F |   | F | F | F  |    | F  |

```
static boolean[] SieveOfEratosthenes(int n) {
```

```
    boolean isprime[] = new boolean[n+1];
```

```
    Arrays.fill(isprime, true);
```

```
    isprime[0] = false;
```

```
    isprime[1] = false;
```

```
    for(int i = 2; i*i <= n; i++) {
```

```
        for(int j = 2*i; j <= n; j+=i)
```

```
            isprime[j] = false;
```

```
    }
```

```
    return isprime;
```

```
}
```

GCD.

$\text{gcd}(a, b) = \text{gcd}(a-b, b);$

$a > b; \leftarrow \text{condition.}$

$\text{gcd}(a, b) = \text{gcd}(b, a \% b)$

$a \% b \neq 0 \leftarrow \text{base case.}$

हाल्लो वाने  
इधर निरव  
हो।

Static int gcd(int a, int b) {  
 if (b == 0)  
 return a;  
 return gcd(b, a % b);  
}

3

### Modulo Arithmetics.

$a^b \rightarrow (a^2)^{b/2}$       b is even  
 $a^b \rightarrow a \cdot a^{b-1}$       b is odd

```

Static int fastPower(int a, int b) {
    int res = 1;
    while (b > 0) {
        if ((b & 1) != 0)
            res *= a;
        a = a * a;
        b = b >> 1;
    }
    return res;
}
    
```

3

### \* modulo property

$$(a + b) \% n = (a \% n + b \% n) \% n$$

$$(a * b) \% n = (a \% n * b \% n) \% n$$

compute  $a^b$  modulo  $10^9 + 7$ ;

static long fastPower(long a, long b, int n) {

long res = 1;

while(b > 0) {

if((b & 1) != 0)

res = (res \* a % n) % n;

a = (a % n \* a % n) % n;

b = b >> 1;

}

return res;

}

\* Trailing zeroes in factorial  $\Rightarrow$  Count 5.  $\Rightarrow (n/5)^k$



No of trailing zeroes

$$= \left\lfloor \frac{n}{5} \right\rfloor + \left\lfloor \frac{n}{25} \right\rfloor + \left\lfloor \frac{n}{125} \right\rfloor + \left\lfloor \frac{n}{625} \right\rfloor$$

Given n

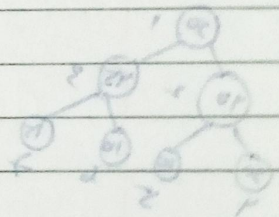
int res = 0;

for(int i = 5; i <= n; i = i \* 5

res = res + n/i;

return res;

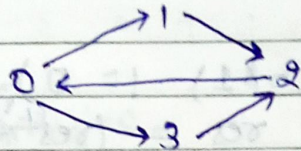
एक शक्ति है  $a^b$  में  
res ऐसा mod है  
आ  $a^b \mod n$



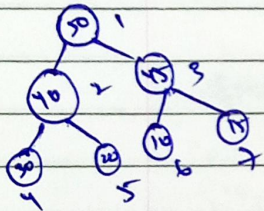
\* In Hashing <sup>space still use kr marte</sup> time Complexity of insert, search, delete is  $O(1)$ .

\* Strongly Connected :- from any point reach all the point.

eg.



Heap  $\leftarrow$  complete binary tree. that satisfies a heap property.



Node  $\rightarrow i$ ,  $\text{parent}(i) = \lfloor \frac{i}{2} \rfloor$

$\text{left\_child}(i) = 2 \times i$ ,  $\text{right\_child} = 2 \times i + 1$

leaf node  $\Rightarrow \lfloor \frac{n}{2} \rfloor + 1$  to  $n$ .

## HASHMAP

Hash table



Node [ ] hashTable = new Node[7];

Node {

~~int~~

k key

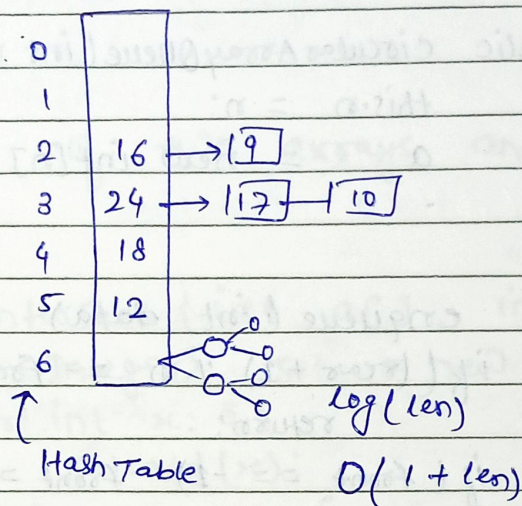
v value

int hash

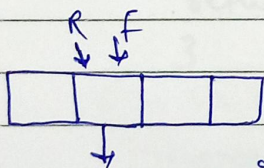
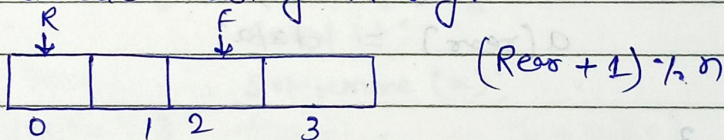
Node next

$$h(\text{key}) = \text{key} \% 7$$

keys = [24, 16, 12, 17, 18, 10, 9]



Queue Using Array.



इस पर pop करने के लिए  
 $R = F = -1$  करेंगे.

isEmpty() { Front = -1  
 Rear = -1 }

isFull()  $[(R + 1) \% n == F]$

$(\text{Front} + 1) \% n$

## Queue Using Array

```
class CircularArrayQueue {  
    int a[], N, front = -1, rear = -1;
```

```
    public CircularArrayQueue(int n) {  
        this.n = n;  
        a = new int[n];
```

3

```
    void enqueue(int data) {  
        if ((rear + 1) % n == front)  
            return;  
        if (front == -1) front = 0;
```

```
        rear = (rear + 1) % n;  
        a[rear] = data;
```

3

```
    void dequeue() {  
        if (front == -1)  
            throw Exception();  
        int result = a[front];
```

```
        if (front == rear)  
            front = rear = -1;
```

else

```
        front = (front + 1) % n;
```

```
        return result;
```

3

3

Q) Union of two arrays without Set. and then return number of distinct element after Union.

Q. Intersection of two arrays. and then return Size.

```
int intersect(int a[], int b[]) {  
    Set<Integer> set = new HashSet<>();  
    for(int x: a)  
        set.add(x);  
    for(int x: b) {  
        if(set.contains(x)) {  
            count++;  
            set.remove(x);  
        }  
    }  
    return count;  
}
```

## 7. Reverse Integers

Given a signed 32-bit integer  $x$ , return  $x$  with its digits reversed. If reversing  $x$  causes the value to go outside the signed 32-bit integer range  $[-2^{31}, 2^{31}-1]$ , then return 0.

Solution:

```
public int reverse(int x) {
```

```
    long rev = 0;
```

```
    while (x != 0) {
```

```
        rev += x % 10;
```

```
        x = (x - x % 10) / 10;
```

```
        if (x != 0)
```

```
            rev *= 10;
```

```
    }
```

```
    if (rev > Integer.MAX_VALUE || rev < Integer.MIN_VALUE)
```

```
        return 0;
```

```
    return (int) rev;
```

```
}
```

3

## Add Two Numbers.

Class Solution {

int carry = 0;

public ListNode addTwoNumbers(ListNode l1, ListNode l2) {

if (l1 == null && l2 == null && carry == 0) {  
return null;

}

int val1 = l1 == null ? 0 : l1.val;

int val2 = l2 == null ? 0 : l2.val;

int sum = val1 + val2 + carry;

carry = sum / 10;

l1 = l1 == null ? null : l1.next;

l2 = l2 == null ? null : l2.next;

ListNode ans = new ListNode(sum % 10, addTwoNumbers(l1, l2));

return ans;

}

## 1641: Counted Sorted Vowel Strings

public int countVowelStrings(int n) {

int a = 1, e = 1, i = 1, o = 1, u = 1;

while (n-- > 1) {

a = a + e + i + o + u;

e = e + i + o + u;

i = i + o + u;

o = u;

u = u;

return a + e + i + o + u;

class Solution {

public List<List<Integer>> combinationSum(int[] candidates,  
int ~~int~~ target) {

List<List<Integer>> result = new ArrayList();

List<Integer> cur = new ArrayList();

backtrack(candidates, 0, target, cur, result);

return result;

3

public void backtrack(int candidates[], int start,  
int target, List<Integer> cur, List<List<Integer>> result)  
{

if (target == 0) {

result.add(new ArrayList(cur));  
return;

else {

for (int i = start; i < candidates.length; i++) {

if (candidates[i] <= target) {

cur.add(candidates[i]);

backtrack(candidates, i, target - candidates[i],

cur, result);

cur.remove(cur.size() - 1);

3

3 3

3

Ques) Given an Integer array nums, find the Contiguous Subarray (Containing at least one number) which has the target sum and return its sum.

```
public int maxSubArray(int nums[]) {  
    int x = nums[0];  
    int sum = nums[0];  
    for (int i = 1; i < nums.length; i++) {  
        sum = Math.max(nums[i], nums[i] + sum);  
        x = Math.max(x, sum);  
    }  
    return x;  
}
```

83. Remove duplicate from Sorted List

```
public ListNode deleteDuplicates(ListNode head) {
```

```
    if (head == null)
```

```
        return null;
```

```
    ListNode list = new ListNode(head.val);
```

```
    ListNode dummy = list;
```

```
    while (head != null) {
```

```
        while (head != null && list.val == head.val) {
```

```
            head = head.next;
```

```
        }
```

```
        if (head == null) break;
```

```
        list.next = new ListNode(head.val);
```

```
        list = list.next;
```

```
        head = head.next;
```

```
    }
```

```
    return dummy;
```

70. You are climbing a staircase - it takes  $n$  steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways you can climb to the top?

```
Map<Integer, Integer> map = new HashMap<>();
```

```
public int climbStairs(int n) {
    if (n == 1)
        return 1;
    if (n == 2)
        return 2;
    if (map.containsKey(n))
        return map.get(n);
    map.put(n, climbStairs(n-1) + climbStairs(n-2));
    return map.get(n);
}
```

3

```
public int rangeBitwiseAnd(int left, int right) {
```

```
    int res = right;
```

```
    while (left < right) {
```

```
        res = right & (right - 1);
```

```
        right = res;
```

```
    }
```

```
    return res;
```

3

```
a = [[4,5],[1,4]]
```

Arrays.sort(a, (v1, v2) → Integer.compare(a[0], a[1]))

↓

[[1,4],[4,5]]

1395. Input: [2, 5, 3, 4, 1]

Output: 3

exp. (2, 3, 4) (5, 4, 1) (5, 3, 1)

```
* Public int numTeams(int[] rating) {
```

```
    if (rating == null || rating.length == 0)
        return 0;
```

```
    int len = rating.length;
    int dp[] = new int[len];
    int count = 0;
```

```
    for (int i = 0; i < len; i++) {
        for (int j = i; j >= 0; j--) {
            if (rating[i] > rating[j]) {
                dp[i]++;
```

```
                count = count + dp[j];
```

```
            }
```

```
        }
```

```
    }
```

```
    dp = new int[len];
```

```
    for (int i = 0; i < len; i++) {
```

```
        for (int j = i; j >= 0; j--) {
```

```
            if (rating[i] < rating[j]) {
                dp[i]++;
```

```
                count = count + dp[j];
```

```
            }
```

```
        }
```

```
    }
```

```
    return count;
```

```
}
```

Question by Anuj Bhaiya

⇒ Hashmap.

- (1) Find the SubArray with the given sum.
- (2) Find the largest SubArray with equal no. of 1 and 0.

[1, 1, 0, 1, 1, 0, 0]

Count the distinct elements in every window of size K.

a = [1, 2, 2, 1, 3, 1, 1, 3]

o/p 2, 3, 3, 2, 2

Allocate minimum Pages. (Books Allocations)

Minimise the maximum numbers of pages read by a student.

5 books → no. of pages in each book  
[ 10 20 5 15 5 ] → no. of student

if (maxHeap.size() < minHeap.size()) k=2.  
maxHeap.add(minHeap.poll());

- \* Find the median of Running Stream of integers
- \* Josephus game problem.

```
P void insertNum(int num) {  
    if (maxHeap.isEmpty() || maxHeap.peek() >= num)  
        maxHeap.add(num)  
    else  
        minHeap.add(num)  
    if (maxHeap.size() > minHeap.size() + 1)  
        minHeap.add(maxHeap.poll());  
}
```

Allocate minimum pages.

```
int minPages(int a[], int k) {  
    int min = maxOf(a), max = sumOf(a), res = 0;  
    while(min <= max) {  
        int mid = (min + max) / 2;  
        if(isfeasible(a, k, mid)) {  
            res = mid;  
            max = mid - 1;  
        }  
        else  
            min = mid + 1;  
    }  
    return res;  
}
```

```
boolean isfeasible(int a[], int k, int res) {  
    int student = 1, sum = 0;  
  
    for(int i = 0; i < a.length; i++) {  
        if(sum + a[i] > res) {  
            student++;  
            sum = a[i];  
        }  
        else  
            sum += a[i];  
    }  
  
    return student <= k;  
}
```



→ Greedy -

2027. Minimum Moves to Convert String

You are given a String  $S$  consisting of 'n' characters which are either 'x' or '0'.

Return the minimum number of moves required so that all characters of  $S$  are converted to '0'.

```
Pub int minimumMoves (String S) {
```

```
    int i = 0;
```

```
    int Step = 0;
```

```
    while (i < S.length()) {
```

```
        if (S.charAt(i) == 'x') {
```

```
            i = i + 3;
```

```
            Step++;
```

```
        }
```

```
        else
```

```
            i++;
```

```
    }
```

```
    return Step;
```

```
}
```

Write a program to find the Smallest integer value 'b' for the given value 'a' if we multiply the digits of 'b' we should get the exact value of 'a'. Result 'b' must contain more than one digit.

Constraints

$1 \leq a \leq 10000$

input = 10

o/p 25

Explanation:-

$2 * 5 = 10$  Hence 25 is the Smallest value for a.

void Smallest(int a) {

int i, j = 10;

int Small[10000];

if(a < 10) {

cout << a + 10;

return;

}

for(i = 9; i > 1; i--) {

while(a % i == 0)

{

a = a / i;

Small[j] = i;

j++;

}

}

if(a > 10) {

cout << "Not Possible";

return;

for(i = j - 1; i >= 0; i--)

cout << Small[i];

}

a=13

Wave array.

$$arr[1] \geq arr[2] \leq arr[3] \geq arr[4] \leq arr[5]$$

$$i/p = 1, 2, 3, 4, 5$$

$$o/p = 2, 1, 4, 3, 5$$

```
int i = 0;
```

```
while(i < n) {
```

```
    if(n % 2 == 1 && i == n - 1)
```

```
        break;
```

```
    int temp = arr[i];
```

```
    arr[i] = arr[i + 1];
```

```
    arr[i + 1] = temp;
```

```
    i = i + 2;
```

```
}
```

```

Static void PrintTwinPrime(int n) {
    boolean prime[] = new boolean[n+1];
    for(int i=0; i<=n; i++)
        prime[i] = true;

```

```

    for(int p=2; p*p<=n; p++) {
        if(prime[p] == true)
            for(int i=p*2; i<=n; i+=p)
                prime[i] = false;

```

3

```

    for(int i=2; i<=n-2; i++)
        if(prime[i] == true && prime[i+2] == true)
            s.o.p(i + " " (i+2));

```

3

LeetCode 560 Subarray Sum Equals K.

```
public int subarraySum(int[] nums, int k) {
```

```
    int count = 0; sum = 0;
```

```
    Map<Integer, Integer> m = new HashMap<>();
```

```
    m.put(0, 1);
```

```
    for(int i = 0; i < nums.length; i++) {
```

```
        sum += nums[i];
```

```
        if (m.containsKey(sum - k))
```

```
            count += m.get(sum - k);
```

```
        m.put(sum, m.getOrDefault(sum, 0) + 1);
```

```
    }
```

```
    return count;
```

```
}
```